



BugIt
by Taskivator

단계별 완전 매뉴얼

BugIt

VS Code용 QA 버그 등록 에이전트

최초 설정 & 일상 사용: 이런 작업을 한 번도 해본 적 없는 분들을 위해 클릭 하나하나까지 상세히 안내하는 가이드입니다. 기술 배경은 필요 없습니다. 순서대로 따라 하시면 오늘 바로 깔끔한 버그 티켓을 등록하실 수 있습니다.

실행 환경: VS Code. Windows (기본), macOS & Linux

번역 안내.

이 문서는 기계 번역되었으며 원어민 검수를 거치지 않았습니다. 기준이 되는 것은 영어판이며, 내용이 다를 경우 영어 원문이 우선합니다. 가장 정확하고 최신인 표현은 영어 문서를 참고하시기 바랍니다.

이 가이드에 담긴 내용

Part 1은 한 번만, 순서대로 진행하세요. 그 이후 부분은 필요할 때마다 펼쳐 보시면 됩니다. 마지막의 **문제 해결**과 **FAQ**가 가장 흔한 질문들에 답하니, 지원팀에 메일을 보내기 전에 먼저 확인해 주세요.

Part 1 · 설정 (한 번만 진행)

Step 0: GitHub Copilot 준비하기

Step 1: VS Code 설치하기

Step 2: BugIt 다운로드 파일 압축 풀기

Step 3: VS Code에서 BugIt 열기

Step 4: 채팅에서 BugIt 에이전트 선택하기

Step 5: 라이선스 활성화하기

Step 6: 약관 동의하기 (한 번만)

Step 7: "Begin setup" 실행하기

Step 8: 추적 시스템 연결하기

Step 9: 로그인 정보 저장하기

Step 10: 준비 상태 확인하기

Part 2 · 일상적인 사용

명령 한눈에 보기

Part 3 · 나만의 것으로 만들기

Part 4 · 업데이트, 백업 & 안전

Part 5 · 문제 해결 & FAQ

자주 묻는 질문

Team 라이선스: 자주 묻는 질문

Part 6 · 라이선스 & 지원

여러분을 안전하게 지켜주는 단 하나의 규칙

BugIt은 미리보기를 보여주고, 되돌릴 수 없는 작업에는 사용자가 직접 **FILE IT** 를 입력하기 전에는 **절대** 아무것도 등록하지 않습니다. 단순히 "예"만으로는 실행되지 않습니다. 항상 여러분이 결정권을 가집니다. 위험 없이 연습해 보고 싶으신가요? **dry run** 이라고 입력하면 보고서 전체를 작성하되 등록하지 **않습니다**.

최초 설정 vs. 일상 사용

Part 1 전체는 한 번만 따라 하시면 됩니다. 그 이후로 BugIt을 여는 일은 순식간이며, 업데이트도 명령 한 번 (**Update**)이면 됩니다. **Part 4**를 참고하세요.

PART 1 · 설정: 한 번만 진행하세요

대략 15분 정도 걸립니다. 각 단계가 이전 단계를 기반으로 하니 순서대로 진행하고, 건너뛰지 마세요.

이 가이드는 GitHub Copilot 경로를 따릅니다

BugIt은 "운전자"이고 AI가 "엔진"입니다(보고서를 실제로 작성하는 것은 AI입니다). 이 단계에서는 VS Code의 GitHub Copilot으로 설정합니다. 처음부터 시작한다면 가장 빠른 방법입니다. BugIt은 Claude 확장, 명령을 실행할 수 있는 다른 어시스턴트, 그리고 직접 준비한 AI 키로 단독 실행하는 방식에서도 작동합니다. 이미 그중 하나를 사용하고 있다면 2단계로 건너뛰고 VS Code 대신 해당 도구에서 BugIt 폴더를 여세요.

무엇인가요: GitHub Copilot은 GitHub(Microsoft 소유)에서 제공하는 AI 어시스턴트입니다. 가벼운 사용을 위한 무료 등급과, 많이 사용하는 경우를 위한 유료 등급이 있습니다. Step 1에서 설치하고 로그인하게 됩니다.

1. **GitHub 계정**이 필요합니다. 없다면 github.com/signup 에서 무료로 만드세요.
2. Copilot 무료 등급으로도 BugIt을 체험하기에 충분합니다. 하루 종일 버그를 등록하신다면 유료 "Copilot Pro" 플랜이 그만한 가치가 있으며, github.com/features/copilot 에서 언제든지 업그레이드할 수 있습니다.
3. 실제로 Copilot을 켜는 작업은 Step 1에서 VS Code 안에서 진행하므로, 지금은 여기서 더 할 일이 없습니다.

VS Code는 Microsoft의 무료 앱입니다. BugIt이 들어 있는 "창"이라고 생각하시면 됩니다.

1. 웹 브라우저를 열고 code.visualstudio.com 으로 이동하세요.
2. 파란색 **Download** 버튼을 클릭하세요(시스템을 자동으로 감지합니다).
3. 다운로드한 파일을 열어 설치하세요. **기본 옵션을 모두 그대로 두고** Next/Install만 계속 클릭하시면 됩니다.
4. 설치가 끝나면 VS Code를 여세요.

VS Code 안에서 GitHub Copilot 켜기

1. VS Code 왼쪽 끝에서 **Extensions** 아이콘(작은 정사각형 4개 모양)을 클릭하세요.
2. 검색창에 **GitHub Copilot**을 입력하세요.
3. "GitHub Copilot"의 **Install**을 클릭하세요(제안된다면 "GitHub Copilot Chat"도 함께).
4. **Sign in to GitHub**를 묻는 창이 뜨면 클릭하고, 열리는 브라우저 창에서 GitHub 계정으로 로그인하세요.

✅ **다음과 같이 보이면 성공입니다:** VS Code 하단에 작은 Copilot 아이콘이 나타나고, Chat 패널이 열립니다(왼쪽의 말풍선 아이콘 클릭 또는 **Ctrl+Alt+I**).

2 BugIt 다운로드 파일 압축 풀기

구매하시면 .zip 파일(예: `bugit-qa-agent.zip`)이 함께 제공됩니다. 압축을 풀어야 하며, 압축된 상태로는 실행되지 않습니다.

1. .zip 파일을 찾으세요(보통 **Downloads** 폴더에 있습니다).
2. **마우스 오른쪽 버튼**으로 클릭 → **Extract All...** 선택(Windows) 또는 더블클릭(Mac).
3. **Documents** 폴더처럼 단순한 위치를 선택한 뒤 Extract를 클릭하세요.
4. 이제 여러 파일이 들어 있는 **BugIt** 폴더가 생깁니다. 위치를 기억해 두세요.

OneDrive나 동기화 폴더에는 넣지 마세요

클라우드 동기화 폴더는 파일 충돌을 일으킬 수 있습니다. Documents나 Desktop이 딱 알맞습니다. 또한 내부 파일 이름을 바꾸지 마시고 폴더를 그대로 두세요.

3 VS Code에서 BugIt 열기

1. VS Code에서 **File** → **Open Folder**(왼쪽 상단 메뉴)를 클릭하세요.
2. BugIt 압축을 푼 위치(예: Documents)로 이동하세요.
3. **BugIt** 폴더를 한 번 클릭해 선택한 뒤 **Select Folder**를 클릭하세요.
4. VS Code가 "Do you trust the authors of the files in this folder?"라고 묻으면 **Yes, I trust the authors**를 클릭하세요.

✔ 다음과 같이 보이면 성공입니다: VS Code 왼쪽에 BugIt 파일들(`tools` , `.github` 같은 폴더와 `README.md` 같은 파일)이 나열됩니다.

4 채팅에서 BugIt 에이전트 선택하기

1. Copilot **Chat** 패널을 여세요. 왼쪽의 말풍선 아이콘을 클릭하거나 `Ctrl+Alt+I` (Windows) / `Cmd+Alt+I` (Mac)를 누르세요.
2. 채팅 상자 상단에 작은 **모드 드롭다운**이 있습니다("Ask" 또는 "Agent"라고 표시될 수 있습니다).
3. 이를 클릭하고 목록에서 **BugIt QA Agent**를 선택하세요.

목록에 "BugIt QA Agent"가 안 보이시나요?

다른 폴더가 아니라 BugIt 폴더를 열었는지 확인하세요(Step 3). 에이전트는 그 폴더 안에서 불러옵니다. 필요하다면 폴더를 닫았다가 다시 여세요.

5 라이선스 활성화하기

BugIt 계정만 있으면 되고, 복사하거나 붙여넣을 것이 없습니다. 활성화는 브라우저에서 이루어집니다.

1. 채팅 상자에 **hi** 를 입력하고 Enter를 누르세요.
2. **Activate** 를 입력하세요. BugIt이 브라우저에서 BugIt Portal을 엽니다. 본인 계정으로 로그인하고, Solo 또는 Team 권한을 선택한 다음 이 기기를 승인하세요.
3. VS Code로 돌아오면 활성화가 자동으로 완료됩니다. 비밀번호는 브라우저에만 머물며 VS Code에는 절대 입력되지 않습니다.

✔ **다음과 같이 보이면 성공입니다:** BugIt이 활성화 완료를 알리고 로그인한 계정을 알려 줍니다.

계정을 비공개로 유지하세요

계정과 권한은 여러분에게 귀속됩니다. 공유하거나, 게시하거나, 재판매하지 마세요. 공유된 접근 권한은 철회될 수 있습니다. Team 구성원은 각자 자신의 계정으로 로그인하며, 공유 로그인도 공유 접근 권한도 없습니다. 나중에 다른 컴퓨터로 옮기는 것은 괜찮습니다(FAQ 참고).

6 약관 동의하기(한 번만)

처음 한 번은 BugIt이 짧은 요약을 보여주고 다른 작업을 하기 전에 동의를 요청합니다.

1. 표시되는 짧은 요약을 읽어보세요(등록되기 전 모든 티켓을 사용자가 검토합니다. 프로젝트의 안전은 사용자의 책임입니다. 안전장치는 보조 수단이지 보장이 아니며, Copilot에서 응답하는 AI 모델을 따라 작동하므로, 가볍고 무료인 모델은 더 강력한 모델과 달리 가끔 단계를 건너뛰거나 명령을 다시 요청할 수 있습니다).
2. BugIt이 보여 주는 두 가지 선택지에서 **I agree**를 선택하세요.

✔ **다음과 같이 보이면 성공입니다:** BugIt이 확인하고 설정을 이어갑니다. 이 질문은 한 번만 받으며, 이후로는 기억합니다.

7 "Begin setup" 실행하기

이제 BugIt이 쉬운 말로 질문하면서 스스로를 설정합니다. 설정 파일을 직접 편집할 일은 전혀 없습니다.

여러분이 입력

Begin setup

짧은 질문들을 하고 선택한 것만 겁니다. 가장 중요한 질문은 추적 시스템입니다.

8 추적 시스템 연결하기

설정은 어떤 추적 시스템을 쓰는지 기록했습니다. 실제로 티켓을 등록할 수 있게 해 주는 단계가 바로 이 단계입니다. 본인 계정에서 API 토큰을 만든 뒤 명령 한 번으로 이 컴퓨터에 저장합니다. BugIt은 그 토큰으로 추적 시스템의 REST API에 직접 등록하므로, 계속 실행해 둘 것이 없고 등록에 MCP 서버를 사용하지 않습니다.

8a · 본인 계정에서 토큰 만들기

사용 중인 추적 시스템	만드는 곳
Jira Cloud	id.atlassian.com/manage-profile/security/api-tokens
Azure DevOps	dev.azure.com/YOUR-ORG/_usersSettings/tokens
GitHub Issues	github.com/settings/personal-access-tokens/new
GitLab Issues	gitlab.com/-/user_settings/personal_access_tokens
Bugzilla	본인의 Bugzilla: Preferences → API Keys
YouTrack	본인의 YouTrack: 아바타 → Profile → Account Security → Authentication
Linear	linear.app/settings/account/security
Shortcut	app.shortcut.com/settings/account/api-tokens
ClickUp	app.clickup.com/settings/apps
Asana	app.asana.com/0/my-apps
Trello	trello.com/apps/admin (API 키와 그것으로 승인한 토큰, 두 가지)

토큰이 표시되는 순간 바로 복사하세요

대부분의 서비스는 새 토큰을 단 한 번만 보여줍니다. 즉시 복사해 BugIt이 저장해줄 때까지(Step 9) 안전한 곳에 보관하세요. 곧 다시 만들지 않아도 되도록 추적 시스템이 제공하는 가장 긴 만료 기간을 선택하세요.

8b · 이 컴퓨터에 저장하기

어시스턴트에게 추적 시스템 연결을 요청하면 대신 실행해 줍니다. 직접 하려면 VS Code에서 터미널을 열고 사용하는 추적 시스템에 맞는 명령을 실행하세요. `jira` 대신 `ado`, `github`, `gitlab` 등을 넣으면 됩니다.

여러분이 실행(터미널에서)

```
python tools/connect.py jira
```

8c · 마스크된 입력창에 토큰 붙여넣기

이 명령은 마스크된 로컬 입력창으로 토큰을 요청합니다. 화면에 표시되지 않으며 채팅이나 티켓, `config.json` 에 절대 닿지 않습니다. 오직 그곳에만 붙여넣으세요. 그러면 BugIt이 로그인해 토큰으로 실제 보이는 프로젝트를 나열하고, 여러분이 만들 수 있는 이슈 유형을 읽은 뒤, **등록할 수 없는 자격 증명은 저장을 거부합니다**. 따라서 설정이 끝났다면 등록도 됩니다.

8d · 연결 확인하기

언제든 확인하려면 터미널에서 점검을 실행하거나 채팅에 물어보세요.

여러분이 실행(터미널에서)

```
python tools/connect.py jira --check
```

여러분이 입력

```
Check status
```

✔ 다음과 같이 보이면 성공입니다: BugIt이 추적 시스템을 연결됨/정상으로 표시합니다.

VS Code를 닫아도 실행해 둘 것이 없습니다

API 토큰은 VS Code가 아니라 운영체제의 자격 증명 저장소에 있습니다. 그래서 VS Code를 다시 열어도 시작할 것이 없고, BugIt은 바로 등록할 수 있습니다. 확인하고 싶을 때는 `python tools/connect.py jira --check` 를 실행하세요.

9 로그인 정보 저장하기

토큰을 다시 찾아볼 필요가 없도록, BugIt은 컴퓨터에 내장된 안전 보관함에 로그인 정보를 저장합니다.

8c 단계에서 마스크된 입력창에 토큰을 붙여넣으면, 컴퓨터 자체가 이를 보호합니다. Windows에서는 그 컴퓨터의 사용자 Windows 계정만 열 수 있는 암호화된 기록으로, macOS와 Linux에서는 키체인이나 시크릿 서비스에 저장됩니다. 일반 파일에는 절대 저장되지 않으며, 설정한 추적 시스템 외에는 어디로도 전송되지 않습니다. 기본적으로 채팅 기록에 절대 닿지 않도록 **마스크된 터미널 입력창**을 통해 받습니다.

여러분이 입력

```
Check saved credentials
```

...그러면 어떤 추적 시스템에 자격 증명에 저장되어 있는지 표시하며, 자격 증명 값 자체는 표시하지 않습니다. BugIt은 저장된 자격 증명 값을 절대 표시하거나 복사해 주지 않습니다.

10 준비 상태 확인하기

실제로 등록하기 전에 모든 것을 다시 확인하도록 BugIt에 요청하세요.

여러분이 입력

Check readiness

아직 빠진 것이 있으면 나열해 줍니다(보통 "추적 시스템을 연결하세요" 또는 "활성화하세요" 정도입니다). 모두 초록볼이 되면 "✅ Setup complete."라는 메시지가 표시됩니다.

이제 설정이 끝났습니다. 영원히

Part 1을 다시 반복할 일은 없습니다. 이제부터 BugIt을 여는 방법은 간단합니다. 폴더 열기 → `hi` 입력. 나중에 어떤 선택이든 바꾸고 싶다면 `Begin setup` 이라고 입력하고 무엇을 바꿀지 말씀하세요.

GitHub Copilot이 없으신가요? 터미널 마법사가 있습니다

Copilot을 사용할 수 없다면 내장 Terminal을 열고(Terminal → New Terminal) `python tools/setup_wizard.py` 를 실행하세요. 몇 가지 질문을 한 뒤 AI 없이도 설정을 작성해 줍니다. 자체 AI 키로 BugIt을 독립 실행할 수도 있습니다. FAQ를 참고하세요.

PART 2 · 일상적인 사용

설정이 끝나면 모든 작업을 채팅에서 진행합니다. 자연스럽게 말하거나 아래 예시를 그대로 사용하세요. 전체 목록을 보려면 언제든지 `help` 를 입력하세요.

버그 보고서 작성하기

BugIt의 주된 역할입니다. 버그를 평범한 문장 하나로 설명하면, 전체 티켓을 작성하고, 버전과 카테고리를 자동으로 채우고, 중복을 확인하고, 미리보기를 보여준 뒤 사용자가 `FILE IT` 을 입력하면 등록합니다. 단순히 "예"만으로는 실행되지 않습니다.

여러분이 입력

Write a bug report: 아이폰에서 저장을 누를 때마다 앱이 꺼져요

제목, 재현 단계, 예상 결과 대 실제 결과, 심각도, 플랫폼을 초안으로 작성하며, 확인하신 심각도를 티켓 본문에 기록합니다. 추적 시스템과 프로젝트가 자체 우선순위 또는 심각도 필드를 지원하는 경우에는 해당 필드에도 값을 설정합니다. GitHub, GitLab, Trello에는 이러한 필드가 없습니다. 등록 전에 수정하고 싶으신가요? 그냥 말씀하세요. 예를 들어 "make the severity high" 또는 "add that it started in the latest build."

퀵 버그(빠른 등록)

흔한 유형(crash, error, missing, layout, slow, stuck, data, visual, audio, blocker)의 경우, 퀵 폼이 일부 질문을 건너 뛰고 카테고리나 우선순위를 대신 채워줍니다. 그러면서 중복 검색은 왕복을 아끼기 위해 건너뛹니다. 이미 새로운 문제라는 것을 알고 있을 때 사용하세요. 미리보기에 그렇게 표시되며, 표준 보고서는 여전히 전체 중복 확인을 실행합니다.

여러분이 입력(예시)

Quick bug: crash on startup after update

Quick bug: layout button overlaps text on the settings screen

Quick bug: blocker can't continue past the payment step

Quick bug: slow the dashboard takes 20 seconds to load

크래시 버그 보고서

크래시 도구(Sentry 등)를 연결했다면, BugIt은 위의 모든 작업에 더해 보고서를 크래시와 매칭하고 스택 추적을 대신 가져옵니다.

여러분이 입력

Write a crash bug report: game crashes when opening the map

검증 & 종료 코멘트

수정 사항을 테스트한 뒤, BugIt이 티켓에 게시할 깔끔한 코멘트를 작성합니다. 코멘트를 작성(및 선택적으로 게시)할 뿐, 티켓의 상태를 바꾸지는 않습니다. 티켓을 닫거나/해결하거나/재오픈하는 것은 여전히 추적 시스템에서 직접 하셔야 합니다.

여러분이 입력	받게 되는 것
Close #123	어떤 시나리오인지(수정 확인됨, 요청에 따라 종료, 설계상 정상, 재오픈...) 물론 뒤 해당 코멘트를 작성합니다.
Write a verification comment for the build you tested on Windows	빌드 세부 정보가 채워진 "수정 확인됨" 코멘트.
Write a reopen comment for the build you tested on Windows	"문제가 여전히 발생함" 코멘트(이후 티켓은 직접 재오픈).

번역

설정된 언어들 사이에서 보고서나 용어를 번역하며, 팀의 정확한 표현(용어집 기반, Part 3 참고)을 사용합니다.

여러분이 입력(예시)

Translate to ja: 프로필 화면에서 저장 버튼이 작동하지 않아요

Translate this bug report to English: [텍스트 붙여넣기]

정보 찾아보기(사양 & 용어집)

사양을 연결했거나 용어집을 채웠다면, 프로젝트에 관해 BugIt에 질문하세요.

여러분이 입력(예시)

What is [용어]?

Walk me through the checkout flow

What's new in specs?

중복 탐지

이는 표준 보고서에서, 아무것도 기록되기 전에 자동으로 이루어집니다. BugIt이 추적 시스템에서 일치하는 티켓(표현이 조금 다른 것까지)을 검색해 아무것도 기록되기 전에 경고하므로 회원님이 판단할 수 있습니다. 이는 보장이 아니라 확인입니다. 검색이 읽는 티켓 수에는 한도가 있어, 찾지 못한 중복은 그대로 등록될 수 있습니다. 쿼리 폼은 왕복을 아끼기 위해 검색을 의도적으로 건너뛰며 미리보기에 그렇게 표시하므로, 둘 중 어느 쪽이었는지 항상 알 수 있습니다.

고급 기능(켜신 경우)

이렇게 입력

하는 일

Triage digest

즉석 스탠드업: 영역/심각도별 미해결 버그, 가장 오래된 항목 표시.

Export bug to file

등록 대신 보고서를 Markdown/CSV/JSON으로 저장합니다(추적 시스템 설정 전에 유용).

Undo last filing

감사 로그의 마지막 쓰기를 보여 줍니다. BugIt은 티켓을 삭제하거나 종료할 수 없으며, 새 미리보기와 **FILE IT** 이후에는 자신이 작성한 코멘트나 첨부 파일만 제거할 수 있습니다.

(자동)

긴급 버그에 SLA 플래그 · 카테고리별 필드.

전체 명령 목록

여러분이 입력

help

...언제든 에이전트 안의 모든 명령을 보여줍니다.

언제든 안전하게 연습하세요

`dry run` 이라고 입력하면 BugIt이 아무것도 등록하지 **않고** 보고서 전체를 작성합니다. 학습이나 테스트에 안성맞춤입니다.

명령 한눈에 보기

원하는 것을 늘 자기 말로 설명해도 됩니다. 정해진 표현이 편하다면, BugIt이 인식하는 문구는 다음과 같습니다.

이렇게 입력

무엇이 일어나나

버그 등록

`Quick bug: [type] [subject]`

빠르게 등록합니다. 중복 검색은 의도적으로 건너뛰며, 미리보기에서 그 사실을 알려 줍니다.

`dry run`

보고서를 전부 작성하고 아무것도 등록하지 않습니다. 안전하게 연습하는 방법입니다.

`FILE IT`

등록을 실행하는 유일한 입력입니다. 메시지 전체로 한 번만 입력하며, 방금 읽은 초안에만 적용됩니다.

`Export bug to file`

보고서를 추적 시스템으로 보내지 않고 파일로 저장합니다.

`Undo last filing`

마지막 기록을 보여주고, BugIt이 추가한 댓글이나 첨부을 제거할 수 있습니다. 티켓을 닫을 수는 없습니다. 감사 로그가 켜져 있어야 합니다.

`Triage digest`

최근에 등록한 내용을 요약해 보여 줍니다.

프로젝트 지식

`What is [term]?`

용어집에서 용어를 찾습니다.

`What's new in specs?`

BugIt에 지정한 명세 폴더의 최근 변경 사항을 보여 줍니다.

`Look up crash [ID]`

충돌 도구를 검색해 링크만 돌려줍니다. 보고서는 작성되지 않습니다.

`Look up #[ticket]`

추적 시스템을 검색해 링크만 돌려줍니다. 보고서는 작성되지 않습니다.

`Seed glossary: [text]`

텍스트에서 용어집 후보를 제안합니다. 하나씩 승인하시면 됩니다.

`Learn from my recent tickets`

이미 추적 시스템에 있는 버그에서 작성 스타일을 배웁니다.

설정과 상태

`Begin setup`

전체 초기 설정, 또는 나중에 항목 하나만 변경합니다.

`Check status`

도구가 연결되어 있나요?

이렇게 입력	무엇이 일어나나
Check readiness	등록하기까지 무엇이 남았나요?
Add a custom MCP server	기본 제공되지 않는 도구를 연결합니다.
Fix servers	MCP 연결을 진단하고 복구합니다.
Check saved credentials	이름만 보여 줍니다. BugIt은 저장된 자격 증명의 값을 결코 표시하지 않습니다.
Talk to me in [language]	이 컴퓨터에서 BugIt이 대화하는 언어를 바꿉니다. 보고서를 작성하는 언어와는 별개입니다.
help	이 표의 짧은 버전을 채팅에 표시합니다.
라이선스, 업데이트, 백업	
Activate	이 컴퓨터를 브라우저에서 승인합니다. 키나 비밀번호를 편집기에 붙여 넣는 일은 없습니다.
License status	요금제, 권한, 오프라인 상태를 보여 줍니다.
What version am I on?	현재 버전과 대기 중인 업데이트가 있는지 보여 줍니다.
Update	최신 서명 버전을 그 자리에서 설치합니다. 설정은 유지되며 먼저 백업됩니다.
Back up my settings	설정, 용어집, 문체, MCP 구성을 로컬에 저장합니다. 자격 증명은 포함되지 않습니다.
List backups	돌아갈 수 있는 백업 목록을 보여 줍니다.
Restore my settings	그중 하나로 되돌립니다.
Create support bundle	지원 메일에 첨부할 수 있도록 민감 정보를 제거한 진단 파일입니다. 자격 증명, 티켓, 경로가 들어가지 않습니다.
Team	
Authorize this Team device	이 컴퓨터를 자신의 BugIt 계정으로 브라우저에서 승인합니다.
Manage my devices	BugIt 포털을 엽니다. 기기 로그아웃과 좌석 반환은 그곳에서 합니다.
Manage my Team	BugIt 포털을 엽니다. 구성원, 역할, 좌석, 라이선스 관리는 그곳에서 합니다.

PART 3 · 나만의 것으로 만들기: 프로젝트 지식 추가하기

기본 상태의 BugIt은 백지 상태의 QA 에이전트입니다. 구매 후 프로젝트 내용을 가르치는 방법을 안내합니다. 코딩은 필요 없고, 대부분 채팅으로 대화하기만 하면 됩니다. 추가한 모든 것은 사용자의 컴퓨터에만 남습니다.

1 · 팀의 용어(용어집)

이를 통해 번역과 카테고리 추정이 프로젝트에 맞게 정확해집니다.

1. 파일 목록에서 `.github/glossary/terms.template.md` 를 여세요.
2. 표에 팀의 용어를 채우세요. 기능명, 화면명, 항목명, 약어, 그리고(두 언어를 사용한다면) 그 번역입니다.
3. 파일을 저장하세요(`Ctrl+S`). 끝입니다. 이제부터 BugIt이 이 정확한 용어를 사용합니다.

단축 방법

`Begin setup` 중 "glossary auto-seed"를 켜면 BugIt이 기존 티켓에서 용어를 제안하므로, 각각을 승인하기만 하면 됩니다.

2 · 버그 스타일(하우스 스타일)

티켓이 팀의 정확한 형식(제목 스타일, 심각도 라벨, 필수 항목)을 따르길 원하시나요? 설명할 필요 없이 **보여주기만** 하면 됩니다.

1. 채팅에서 "match my team's style"라고 말하세요.
2. 추적 시스템의 기존 티켓 **스크린샷 하나**를 붙여넣으세요.
3. 이미지는 이미 대화에 참여 중인 AI 모델이 해당 제공업체의 약관에 따라 읽으며, BugIt은 거기서 추출한 텍스트에서 개인 정보를 제거하고 형식을 저장해 이후 모든 티켓에 이를 따릅니다.

3 · 카테고리, 플랫폼 & 필드

BugIt에 쉬운 말로 말하기만 하면 설정이 대신 업데이트됩니다.

여러분이 입력(예시)

```
My categories are Gameplay, UI, Audio, and Networking
We test on Windows and Xbox
```

4 · 나만의 도구(사용자 지정 연결)

기본 제공 목록에 없는 도구를 사용하시나요? 기본 제공 도구와 똑같이 연결할 수 있습니다.

여러분이 입력

```
Add a custom MCP server
```

짧은 이름과 주소(<https://> 여야 함)를 알려주면, BugIt이 연결하고 상태를 점검해 줍니다.

5 · 진행하면서 그냥 고쳐주기

가장 쉬운 방법입니다. BugIt이 티켓 초안을 작성하면 마음에 들지 않는 부분(레이블, 표현, 심각도)을 고쳐주세요. "learn from your edits"를 켜두면(권장) 선호도를 기억해 다음번에 반영합니다. 프로젝트 지식이 사용자의 컴퓨터에 자연스럽게 쌓이며, 그중 어느 것도 Taskivator로 전송되지 않습니다.

추가한 모든 것은 사용자의 것입니다

용어집, 하우스 스타일, 카테고리, 학습된 선호도는 사용자의 컴퓨터에 저장되고, 모든 업데이트 전에 백업됩니다. BugIt이 요청을 수행할 때 관련 내용은 사용자가 연결한 AI 모델과 도구로 전송될 수 있습니다. 그중 어느 것도 Taskivator로 전송되지 않으며, 공유 번들은 사용자가 보내기로 선택할 때만 컴퓨터를 떠납니다.

6 · 팀과 설정 공유하기(Team 라이선스)

용어집, 하우스 스타일, 추적 시스템 선택을 원하는 대로 설정했다면, 각 컴퓨터마다 1~3단계를 반복하는 대신 명령 하나로 팀 전체에 동일한 설정을 전달하세요.

여러분이 실행(한 번, 설정을 마친 뒤)

```
python tools/share.py export
```

이 명령은 비밀번호나 토큰 없이 추적 시스템/크라시/커뮤니케이션 선택, 용어집, 하우스 스타일을 함께 묶은 [config.share.zip](#) 을 만듭니다. 팀에 전달하세요.

각 팀원이 실행

```
python tools/share.py import config.share.zip
```

팀원들은 사용자와 정확히 같은 용어, 버그 스타일, 추적 시스템 설정을 즉시 받습니다. 카테고리를 다시 입력하거나 하우스 스타일을 위해 스크린샷을 다시 붙여넣을 필요가 없습니다. 각자의 라이선스와 기기 설정은 그대로 유지됩니다.

가져오기가 데이터 전송 위치를 바꾸는 경우

공유된 설정이 새로운 사용자 지정 연결이나 알림 주소를 추가한다면, BugIt은 그 어떤 것도 조용히 적용하지 않고 무엇이 바뀌는지 먼저 정확히 보여줍니다. 또한 가져오기 전에 팀원의 현재 설정을 먼저 스냅샷으로 저장하므로, 가져온 결과가 예상과 다르면 [Restore my settings](#) 로 되돌릴 수 있습니다.

PART 4 · 업데이트, 백업 & 안전

업데이트 받기(명령 한 번)

새 버전이 있으면 BugIt이 **hi** 라고 말할 때 한 번 알려줍니다. BugIt 라이선스가 활성화 상태인 동안 소프트웨어 업데이트가 포함됩니다. 설치하려면 다음과 같이 입력하세요.

여러분이 입력

Update

1. BugIt이 먼저 파일의 최신 백업을 만듭니다.
2. 새 버전을 다운로드하고 진짜이며 변조되지 않았는지 확인합니다(암호로 서명되어 있어 가짜이거나 변조된 업데이트는 거부됩니다).
3. 설정, 용어집, 하우스 스타일, 라이선스, 토큰을 건드리지 않고 설치합니다.
4. VS Code를 다시 불러오도록 요청합니다(**Ctrl+Shift+P** → "Reload Window" 입력 → Enter). 새 채팅을 시작하면 새 버전이 적용됩니다.

폴더를 삭제할 일은 절대 없습니다

최초 설치와 달리 업데이트는 제자리에서 이루어집니다. 무언가를 삭제하거나 다시 다운로드할 필요가 전혀 없으며, 설정은 항상 보존되고 백업됩니다.

직접 편집해도 안전한 것, 그리고 안전하지 않은 것

안전하며, 모든 업데이트를 거쳐도 유지됩니다: `config.json`, `.github/instructions/house-style.instructions.md` (Part 3 참고. 동작을 커스터마이징하는 공식 지원 방법입니다), `.github/glossary/terms.template.md`, `.github/instructions/learned.instructions.md`, `.vscode/mcp.json`.

안전하지 않음: 직접 편집하지 마세요

에이전트 파일(`.github/agents/bugit-qa-agent.agent.md`), `.github/instructions/` 아래의 다른 모든 파일, 그리고 `tools/`안의 모든 것은 사용자의 설정이 아니라 제품 자체입니다. 업데이트가 이를 **조용히 덮어쓰며**, 위의 파일들과 달리 **복원할 백업이 없습니다**. BugIt은 시작할 때와 업데이트 설치 직전에 이를 확인하여 변경 사항을 발견하면 경고하지만, 가장 안전한 규칙은 그냥 이 파일들을 수정하지 않는 것입니다.

백업 & 복원

여러분이 입력

하는 일

Back up my
settings

설정, 용어집, 스타일, MCP 엔드포인트, 학습 내용의 로컬 스냅샷을 저장합니다. 서명된 기기 권한과 자격 증명 값은 제외됩니다.

List backups

저장된 모든 스냅샷을 날짜와 함께 보여줍니다.

Restore my settings

스냅샷으로 되돌립니다(그리고 되돌리기 전에 현재 상태를 먼저 스냅샷으로 저장하므로 복원 자체도 취소할 수 있습니다).

모든 업데이트 전 자동 실행

BugIt은 새 버전을 설치하기 전 항상 백업하므로, 업데이트로 설정을 잃을 일은 없습니다. 뭔가 이상하면 **Restore my settings** 로 바로잡으세요.

안전: 보호되는 것**✔ 동의 없이는 아무것도 하지 않음**

모든 티켓과 코멘트는 미리보기로 제공되며, 되돌릴 수 없는 작업에는 **FILE IT** 를 입력해야 합니다. 단순히 "예"만으로는 실행되지 않습니다.

🧪 드라이런 = 쓰기 없음

dry run 이라고 말하면 추적 시스템에 전혀 접속하지 않습니다. 읽기도 포함됩니다. 드라이런은 외부 티켓, 코멘트, 편집, 첨부, 제거, 알림을 모두 차단하고(백업이나 내보내기처럼 직접 실행하는 로컬 명령은 여전히 로컬 파일을 만들 수 있습니다) 검색도, 중복 확인도, 연결 확인도 하지 않습니다. 각각이 사용자의 보드에 접속하고 저장된 토큰을 해제하기 때문입니다. 터미널의 **QA_AGENT_DRY_RUN=1** 이 번들 명령이 코드로 강제하는 스위치입니다. 채팅에서 "dry run"이라고 말하는 것은 어시스턴트에게 자체를 요청하는 것으로, 유용하지만 같은 보장은 아닙니다.

🔍 파일에 비밀 정보 없음

토큰은 OS 저장소에 있으며, 일반 파일에는 절대 없고, 저희에게 전송되지도 않습니다.

🖥️ 사용자의 컴퓨터에서 실행

사용자의 작업물은 연결한 도구와 사용자 자신의 AI 모델에만 전달됩니다.

데이터 & 개인정보 보호

BugIt이 Taskivator로 전송하는 유일한 것은 라이선스/업데이트 데이터입니다. 브라우저에서 Portal을 통해 이루어지는 BugIt 계정 로그인, 단방향으로 해시된 기기 ID(사용자를 식별할 수 없는 뒤섞인 코드), 무작위 설치 식별자, 기기 이름(호스트 이름)과 운영체제 이름, BugIt 버전과 사용자가 고른 요금제 필터, 수명이 짧은 암호 챌린지 정보, 브라우저가 연 Portal 링크에 들어 있는 일회용 승인 토큰, 활성화마다 생성되는 확인 비밀번호(그 해시, 그리고 이 기기의 접근을 해제할 때만 값 자체), 그리고 이 기기에 대해 승인하는 Solo 또는 Team 권한입니다. 붙여넣을 것이 없으며, 비밀번호는 브라우저에만 머뭅니다. 버그 보고서, 사양, 용어집, 설정, 토큰은 절대 Taskivator로 전송되지 않습니다. 등록된 내용은 사용자가 보낸 곳, 즉 사용자 계정의 추적 시스템으로 사용자의 자격 증명을 써서 전달됩니다. 호스팅형 AI 모델을 쓰면 초안은 그 제공업체에도 전달됩니다. 텔레메트리도, 분석도 전혀 없습니다. 자세한 내용은 BugIt 폴더 안의 **PRIVACY.md** 파일을 참고하세요.

위험 & 중요한 주의 사항: 꼭 읽어주세요

AI는 틀릴 수 있습니다

BugIt은 AI 모델로 보고서를 작성하며, 세부 사항을 잘못 이해하거나 없는 내용을 지어낼 수 있습니다. 등록을 확정 (FILE IT 입력)하기 전 항상 미리보기를 읽어보세요. 등록되는 내용을 승인하는 것은 사용자입니다.

실제 추적 시스템에 등록됩니다

승인하면 실제 티켓이 생성됩니다. 연습 중이신가요? `dry run` 을 사용하면 등록 없이 보고서만 작성됩니다.

프로젝트의 안전은 사용자의 책임입니다

BugIt을 어떻게 사용하는지, 그리고 프로젝트·데이터·추적 시스템의 안전은 사용자의 책임입니다. 안전장치는 위험을 줄여주지만 사용자의 검토를 대신하지는 않습니다.

BugIt이 하지 않는 것

BugIt은 이름을 밝힌 열한 개 추적 시스템 모두에, 사용자가 자신의 계정에서 만든 자격 증명으로 등록합니다. 저장하기 전에 BugIt이 그 자격 증명을 선택한 등록 대상에 대해 확인합니다. 정렬 가능한 우선순위 필드에 무엇이 들어가는지는 시스템마다 다르고 GitHub, GitLab, Trello에는 그런 필드가 없어 심각도가 티켓 본문에 기록됩니다. Confluence, Sentry, Notion은 읽기 전용이고, MCP 서버는 그것들에만 필요합니다. Slack, Teams, Discord, 이메일 알림은 기본 기능입니다. 모델을 함께 제공하지 않고 사용자의 AI(Copilot 또는 본인의 OpenAI/Anthropic 키)를 씁니다. 가림 처리는 최선의 노력입니다. 실행 환경은 VS Code, Claude 확장, 일반 터미널입니다. 라이선스 하나 = 기기 하나(Solo) 또는 구성원 5좌석에 1인당 기기 1대(Team). 한 팀은 Team 라이선스를 여러 개 가질 수 있고, 그 좌석은 합산됩니다. 결과와 일관성은 어떤 AI 모델이 답하는지에 좌우됩니다. 더 강력한 모델일수록 아주 가볍거나 무료인 모델보다 안전 절차를 안정적으로 따릅니다.

PART 5 · 문제 해결 & FAQ

문제가 생기면 먼저 쉬운 말로 어시스턴트에게 설명하세요. 설정과 일상 사용에서 생기는 문제 대부분은 편집기 안에서 바로 진단할 수 있습니다.

✦ 먼저 이것을 시도해 보세요: AI에게 고쳐달라고 요청하기

설정 중이든 일상 사용 중이든 무언가 잘못되면, 가장 빠른 해결책은 거의 항상 채팅에서 **어시스턴트에게 바로 요청하는 것**입니다. 무슨 일이 있었는지 쉬운 말로 설명하고(예: "설정이 중간에서 멈췄어요" 또는 "추적 시스템이 연결되지 않아요") 고쳐달라고 요청하세요. AI는 대부분의 문제를 즉시 진단하고 해결할 수 있습니다. 저희에게 메일을 보내기 전에 이 방법을 먼저 시도해 보세요. 대부분의 문제가 몇 초 만에 해결됩니다.

AI 수정 적용하기: "Keep" 버튼

어시스턴트가 파일을 변경해 문제를 고치면, VS Code에 작은 Keep / Undo 표시줄이 나타납니다. 요청한 수정 사항이 마음에 들면 **Keep**을 클릭하세요. 변경 사항은 오직 사용자 자신의 컴퓨터 사본에만 적용됩니다. 취소하려면 **Undo**를 클릭하세요. Keep한 내용은 누구와도 공유되지 않습니다.

표시되는 내용	해야 할 일
"Activate to start"	<code>Activate</code> 를 입력하세요. BugIt Portal이 브라우저에서 열립니다. 로그인하고, Solo 또는 Team 권한을 선택한 다음 이 기기를 승인하세요. 아직 계정이 없으신가요? 스토어에서 받으세요.
브라우저가 열리지 않거나 활성화가 완료되지 않음	<code>Activate</code> 를 다시 입력해 BugIt Portal을 다시 여세요. 그런 다음 로그인과 이 기기 승인을 완료하세요. 활성화는 전적으로 브라우저에서 이루어지므로 붙여넣을 것이 없습니다.
"No tracker enabled"	<code>Begin setup</code> 이라고 입력하고 추적 시스템을 선택하세요.
등록이 거부됨	<code>python tools/connect.py jira --check</code> 를 실행하세요. 추적 시스템이 사용자를 누구로 보는지, BugIt이 무엇을 만들 수 있는지 알려 주고 <code>fixes</code> 에 구체적인 해결 방법을 나열합니다.
계속 활성화를 요청함	<code>Activate</code> 를 입력하고 브라우저 로그인을 완료하세요. BugIt은 저장된 자격 증명 값을 절대 표시하지 않습니다.(처음이신가요? Step 8c 참고.)
버그를 등록하지 않음	<code>Check readiness</code> 라고 입력하세요. 정확히 무엇이 빠졌는지 알려줍니다(보통 추적 시스템 토큰이 아직 저장되지 않은 경우입니다).
업데이트 후 뭔가 이상해 보임	<code>Restore my settings</code> 라고 입력해 되돌리세요.
에이전트가 "지시를 벗어난" 것 같음	<code>Read and follow all your given instructions</code> 라고 입력하거나 새 채팅을 시작하세요. 가볍고 무료인 AI 모델에서 더 자주 발생할 수 있습니다. Copilot의 모델 선택기에서 더 강력한 모델을 선택하면 더 일관됩니다.
답변이 일반적이고 프로젝트에 맞지 않는 느낌	더 많은 맥락을 주거나, <code>Begin setup</code> 중에 기존 티켓 하나를 보여줘 스타일을 학습시키세요. <code>Begin setup</code> 을 다시 실행해 다듬으세요.
설정 선택을 바꾸고 싶음	<code>Begin setup</code> 이라고 입력하고 무엇을 바꿀지 말씀하세요(예: "add a tracker" 또는 "start over"). 해당 부분만 다시 엽니다. 설정이 단순히 중간에 중단되었다면, 그냥 <code>Begin setup</code> 만 입력해도 처음부터 다시 시작하지 않고 이어서 진행합니다.

내장 상태 점검

`Check status` (도구가 연결되어 있나요?) · `Check readiness` (등록 전에 남은 것은?). 더 깊이 확인하려면 Terminal → New Terminal을 열고 `python tools/selftest.py` 를 실행하세요.

자주 묻는 질문

코딩을 알아야 하나요?

아니요. 앱을 설치하고 문장을 입력할 수 있다면 BugIt을 사용할 수 있습니다. 기술적인 부분은 BugIt이 대신 처리합니다.

제가 모르는 사이에 버그가 등록되나요?

절대 그렇지 않습니다. 모든 티켓과 코멘트는 미리보기로 제공되며, 되돌릴 수 없는 작업에는 **FILE IT** 를 직접 입력해야 합니다. 단순히 "예"만으로는 실행되지 않습니다. **dry run** 을 사용하면 쓰기 없이 연습할 수 있습니다.

매번 서버를 시작해야 하나요?

아니요. 등록은 **python tools/connect.py** 로 저장한 API 토큰을 사용하므로 시작할 것이 없습니다. MCP 서버는 Confluence, Sentry, Notion 같은 선택 사항 읽기 전용 연동에만 필요합니다.

두 대의 컴퓨터에서 사용할 수 있나요?

Solo는 한 번에 한 대입니다. Team에서는 구성원마다 자신의 계정과 자신의 기기를 쓰며, 좌석 수는 팀이 보유한 Team 라이선스에서 나옵니다. Team 라이선스 하나가 5좌석을 더합니다. 새 기기로 옮기는 것은 괜찮습니다. 새 기기를 활성화하면 기존 기기는 즉시 갱신할 수 없게 됩니다. 그 자리는 기존 기기가 다시 연결되어 접근 권한을 넘겼음을 확인하거나 현재의 72시간 오프라인 라이선스가 만료될 때 비워지므로, 고정된 대기 시간은 없습니다. **License status** 라고 입력해 확인하세요.

라이선스 서버가 다운되면 어떻게 되나요?

온라인으로 최초 활성화한 후에는 캐시된 라이선스로 최대 72시간 오프라인으로 계속 작업할 수 있으며, 이후에는 온라인 검증이 필요합니다. 덕분에 서버 장애나 인터넷 없는 주말에도 그 시간 안에서는 근무 도중 여러분을 갑자기 막는 일이 없습니다.

GitHub Copilot이 꼭 필요한가요?

가장 쉬운 방법일 뿐입니다. 없다면 Terminal에서 **python tools/setup_wizard.py** 를 실행해 설정하세요. BugIt은 자체 OpenAI/Anthropic 키로 독립 실행도 가능합니다(**python standalone/run.py**).

제 데이터는 비공개로 유지되나요?

네. Taskivator로 전송되는 것은 라이선스/업데이트 데이터뿐입니다: 브라우저의 Portal에서 이루어지는 BugIt 계정 로그인, 단방향 해시된 기기 ID, 무작위 설치 식별자, 기기 이름(호스트 이름)과 운영체제 이름, BugIt 버전과 선택한 요금제 필터, 수명이 짧은 암호 챌린지 정보, 브라우저가 연 Portal 링크에 들어 있는 일회용 승인 토큰, 활성화마다 생성되는 확인 비밀번호(그 해시, 그리고 이 기기의 접근을 해제할 때만 값 자체), 그리고 이 기기에 대해 승인하는 Solo 또는 Team 권한입니다. BugIt 소프트웨어는 Google Ads 측정을 사용하지 않으며 제품 텔레메트리도 전송하지 않습니다. 붙여넣을 것이 없으며 비밀번호는 브라우저에만 머뭅니다. 티켓 자체는 연결한 AI 모델과 추적 시스템에만 전달되며, 저희에게는 절대 전달되지 않습니다.

BugIt의 파일을 직접 수정할 수 있나요?

사용자가 관리하도록 되어 있는 부분, 즉 용어집, 하우스 스타일, 설정(Part 3)은 자유롭게 커스터마이징하시면 됩니다. 다만 라이선스/활성화를 비활성화하지는 마세요. 도구 자체에서 실제 버그를 발견하셨다면, 다음 업데이트에서 모두를 위해 수정될 수 있도록 지원팀에 메일을 보내주세요.

어떤 버그 추적 시스템과 함께 쓸 수 있나요?

열한 곳이며 BugIt은 그 모두에 등록합니다: Jira Cloud, Azure DevOps, GitHub Issues, GitLab Issues, Bugzilla, YouTrack, Linear, Shortcut, ClickUp, Asana, Trello. 모두 안내형 설정이 있고, 모두 사용자가 자신의 계정에서 만든 자격 증명으로 기록하며, 저장하기 전에 BugIt이 그 자격 증명을 등록 대상에 대해 확인합니다. 달라지는 것은 부가 기능입니다. BugIt은 GitHub, GitLab, Bugzilla에 파일을 올리지 않으며 시도하기 전에 그렇게 알려줍니다(GitHub에는 자격 증명으로 인증하는 업로드가 아예 없고, GitLab과 Bugzilla에는 있지만 BugIt이 인증하지 않았습니다), ClickUp은 파일을 받고 지우지 못하며, GitHub, GitLab, Trello에는 우선순위 필드가 없어 심각도가 티켓 본문에 기록됩니다. 설정할 때 고르고, **Begin setup** 으로 언제든지 바꾸세요.

등록 전에 중복 버그를 찾아주나요?

네, 표준 보고서에서는 찾아줍니다. 무언가 작성되기 전에 추적 시스템에서 유사한 보고서(표현이 조금 다른 것까지)를 검색해 일치 항목을 보여주므로 같은 버그를 두 번 등록하지 않을 수 있습니다. 진행 여부는 여전히 사용자가 결정합니다. 퀵 폼은 왕복을 아끼기 위해 그 검색을 의도적으로 건너뛰며, 미리보기에 그렇게 알려 줍니다.

두 개 이상의 언어로 티켓을 작성할 수 있나요?

네. 설정 중 두 번째 언어를 추가하면 모든 보고서가 두 언어로 각각 별도 블록에 작성되며, 용어집을 사용해 제품 용어가 정확하게 유지됩니다. 번역을 임의로 지어내지 않습니다.

스크린샷을 첨부할 수 있나요?

BugIt에 첨부를 요청하세요. 증거 폴더를 만들어 열고 정확한 경로를 알려 줍니다. 그 폴더에 파일을 넣고 끝났다고 알려 주세요. BugIt은 텍스트 파일에 자격 증명이 있는지 확인하고, 있으면 첨부하지 않고 멈춥니다. 보내기 전에 미리보기도 보여 줍니다. 이미지 안은 볼 수 없으므로 스크린샷은 승인하기 전에 직접 확인해 주세요. 첨부는 전용 **FILE IT** 을 받은 뒤에 이루어집니다. 채팅에 붙여넣은 파일에는 BugIt이 접근할 수 없어서 항상 폴더를 안내합니다.

보고서에서 비밀번호와 토큰을 숨겨주나요?

정보 삭제 기능이 켜져 있으면, 등록 전 모든 초안에서 이메일, 토큰, IP 주소를 제거합니다. 미리보기에서는 항상 전체 텍스트를 먼저 확인할 수 있습니다.

실수로 티켓을 등록했어요. 취소할 수 있나요?

미리보기와 직접 입력하는 확인(**FILE IT**)이 대부분의 실수를 사전에 막아줍니다. 감사 로그를 켜두셨다면 **Undo last filing** 이라고 입력하세요. BugIt이 무엇을 기록했는지 보여주고, BugIt이 직접 추가한 댓글이나 첨부를 되돌릴 수 있습니다. 티켓을 닫거나 삭제하는 것은 언제나 추적 시스템에서 사용자가 직접 하는 일입니다. 어느 추적 시스템도 BugIt에 그 방법을 제공하지 않기 때문입니다.

버그를 검증하거나 종료하는 데 도움을 줄 수 있나요?

네. 검증 코멘트를 요청하면 기존 티켓에 명확한 통과/실패 메모(설정된 언어로)를 작성해 주며, 게시 전에 승인하실 수 있습니다.

두 개 이상의 프로젝트에 사용할 수 있나요?

각 BugIt 폴더는 하나의 프로젝트 추적 시스템에 맞게 설정됩니다. 다른 프로젝트라면 **Begin setup** 을 다시 실행해 새 위치를 지정하거나, 프로젝트마다 별도의 사본을 유지하세요.

업데이트는 어떻게 받나요?

새 버전이 나오면 채팅을 시작할 때 BugIt이 알려줍니다. **update** 라고 입력하면 제자리에 설치됩니다. 용어집, 하우스 스타일, 설정은 유지되며, 먼저 백업이 이루어집니다.

라이선스가 만료되면 어떻게 되나요?

기간이 끝나면 본인 계정에서 갱신하세요. 기기는 다음 온라인 확인 시 갱신된 권한을 적용합니다. 72시간 오프라인 라이선스는 이와 별개이며, 만료된 기간이 아니라 일시적인 라이선스 서버 장애만을 다룹니다. 언제든 **License status** 라고 입력하세요.

갱신하거나 다른 라이선스를 구매하면 일수가 누적되나요?

일수는 결코 누적되지 않습니다. 갱신은 현재 권한을 대체합니다. 기간이 새로 시작되고 사용하지 않은 일수는 이월되지 않으므로, 기간이 끝날 무렵에 갱신하세요. **좌석은 다릅니다.** Team에서는 두 번째 Team 라이선스가 자신의 5좌석을 먼저 있던 좌석에 더하며, 각각 자기 기간을 가집니다. 즉 라이선스 두 개면 10좌석입니다. 아래 Team 관련 질문을 참고하세요.

Team 라이선스: 자주 묻는 질문

이 페이지의 내용은 모두 편집기가 아니라 브라우저의 **BugIt 포털 Team** 페이지에서 이루어집니다. Solo 라이선스를 구매하셨다면 이 페이지는 건너뛰어도 됩니다.

Team 라이선스로 무엇을 받나요?

1년간 사용할 5좌석이며, 자동 갱신이 없는 일회성 구매입니다. 팀의 각 구성원은 자신의 BugIt 계정으로 로그인하고 자신의 기기에서 BugIt을 활성화합니다. 공유 키도 공유 로그인도 없습니다. 구성원, 초대, 역할, 좌석은 모두 포털의 Team 페이지에서 관리합니다.

사람을 어떻게 추가하나요?

Team 페이지를 열고 상대방의 이메일 주소로 초대를 보내세요. 여러 주소를 한 번에 붙여넣을 수도 있습니다. 초대는 보낸 주소에서만 유효하므로 전달된 링크를 다른 사람이 쓸 수 없습니다. 대기 중인 초대는 수락되거나 취소될 때까지 좌석을 차지하므로, 모두가 합류하기 전에 팀이 가득 찰 수 있습니다.

누가 무엇을 할 수 있나요?

소유자는 결제와 팀 이전을 포함해 모든 것을 할 수 있습니다. **관리자**는 구성원을 초대하고 제거하며 팀의 공유 기본값을 바꿀 수 있습니다. **구성원**은 BugIt을 사용하고 언제든 나갈 수 있습니다. 소유권은 제거할 수 없고 이전만 가능합니다.

다섯 명이 넘습니다. Team 라이선스를 하나 더 살 수 있나요?

예. 두 번째 Team 라이선스를 구매한 뒤 Team 페이지에서 같은 팀에 추가하세요. **좌석은 합산됩니다.** 두 개면 10좌석, 세 개면 15좌석입니다. 모든 좌석이 보유한 라이선스에서 나오므로 고정된 상한은 없습니다. 추가하면 즉시 적용됩니다.

두 번째 Team 라이선스를 추가하면 팀은 언제 만료되나요?

각 라이선스는 자기 종료일을 유지하며 그때까지 자기 5좌석을 부담합니다. 팀은 라이선스 **가운데 하나라도** 유효한 동안 계속 동작하므로, 팀의 종료일은 둘 중 더 늦은 날짜입니다. 먼저 끝나는 라이선스가 만료되면 팀은 그 라이선스가 부담하던 5좌석

을 잃지만 팀 자체가 닫히지는 않습니다. 두 번째 라이선스가 첫 번째를 연장하지는 않습니다. 한쪽에 석 달이 남았을 때 새 1년짜리를 추가하면 석 달 동안 10좌석, 이후 아홉 달 동안 5좌석이 됩니다. 10좌석을 유지하려면 먼저 끝나는 라이선스를 갱신하세요. Team 페이지에는 각 라이선스가 자기 종료일과 함께 표시되므로 어느 것이 먼저 끝나는지 언제든 볼 수 있습니다.

좌석보다 사람이 많아지면 어떻게 되나요?

라이선스가 끝나기 30일 전, 7일 전, 1일 전에 팀에 알려 드리므로 갑작스러운 일은 없습니다. 그래도 부담하는 좌석보다 많은 좌석을 쓰는 상태에서 라이선스가 끝나면, 먼저 대기 중인 초대가 좌석을 내놓고 그다음에는 가장 최근에 합류한 사람부터 접근 권한을 잃습니다. 소유자는 결코 제거되지 않으므로 갱신할 수 있는 사람이 항상 남습니다. 이미 기록된 내용은 삭제되지 않고 BugIt 접근만 중단됩니다.

팀 소유자가 회사를 떠나면 어떻게 하나요?

소유자는 Team 페이지에서 **후임자**를 지명할 수 있습니다. 이후 소유자와 연락이 되지 않으면 후임자가 인수를 요청할 수 있습니다. 팀 전원에게 알려지고, 소유자가 그전에 중단하지 않는 한 7일 뒤에 적용됩니다. 연락이 되는 소유자는 언제든 팀을 직접 이전할 수 있으며, 그 전에 2단계 인증으로 본인 확인을 요청받습니다.

팀 전체가 같은 방식으로 쓸 수 있나요?

예. BugIt을 원하는 대로 설정한 뒤 `python tools/share.py export` 를 실행하고, 각 동료가 보내 준 파일에 `import` 를 실행하면 됩니다(3부 6절). Team 페이지에는 팀이 합의한 추적 시스템, 프로젝트, 담당자, 라벨도 기록되어 모두가 한곳에서 확인할 수 있습니다. **토큰이나 비밀번호는 절대 넣지 마세요.** 이 값들은 팀 전원이 읽을 수 있습니다. 실제 자격 증명은 언제나 각자의 컴퓨터에서 `python tools/connect.py` 로 개인별로 저장합니다.

누군가 팀을 떠났습니다. 그 좌석은 어떻게 되나요?

Team 페이지에서 제거하세요. 그 사람의 기기는 로그아웃되고 좌석은 바로 다음 초대에 쓸 수 있게 됩니다. 구성원이 스스로 나갈 수도 있습니다. 어느 쪽이든 본인의 BugIt 계정은 그대로 남고 귀하의 팀에 대한 접근만 끝납니다.

영수증과 갱신 안내는 어디로 오나요?

기본적으로 소유자 계정 주소로 갑니다. Team 페이지에서 소유자가 회사명과 별도의 청구 이메일을 추가할 수 있으며, 그 팀의 안내와 영수증은 그곳으로 갑니다. 팀 소유자가 바뀌는 것은 아닙니다.

PART 6 · 라이선스 & 지원

라이선스 약관(쉬운 말 요약)

전체 구속력 있는 약관은 BugIt 폴더의 `LICENSE` 파일에 있으며, 그 문서가 우선합니다. 요약하면 다음과 같습니다.

주제

쉬운 말로

라이선스, 소유가 아님	BugIt을 사용할 라이선스를 구매하는 것이지 소유권이 아닙니다. 모든 권리는 Taskivator에 있습니다.
-----------------	---

주제	쉬운 말로
사용 좌석	Solo = 한 번에 기기 1대. Team = Team 라이선스마다 구성원 5좌석, 1인당 기기 1대이며, 한 팀을 여러 Team 라이선스가 함께 지원할 수 있습니다. 계정과 권한은 개인용이므로 공유, 재판매, 게시하지 마세요.
철회	공유되거나, 환불되거나, 지불 거부되거나, 오용된 접근 권한은 철회될 수 있습니다.
커스터마이징은 가능하지만...	config, 용어집, 템플릿은 자유롭게 편집하되, 라이선스/활성화를 비활성화하거나 우회하지 마세요.
사용자의 책임	BugIt을 어떻게 사용하는지와 프로젝트의 안전은 사용자의 책임입니다. 등록되기 전 모든 보고서를 사용자가 검토하고 승인합니다. 안전장치는 보조 수단이지 보장이 아닙니다.
"있는 그대로", 책임	있는 그대로 제공되며 보증이 없습니다. 법이 허용하는 한도 내에서, 책임은 지불한 금액으로 제한되며 간접적/결과적 손해는 제외됩니다.
기간 & 준거법	구매한 날부터 시작되는 1년 라이선스이며, 1회 구매로 자동 갱신이 없고, 일본 법률의 적용을 받습니다. 환불은 구매한 스토어를 통해 처리됩니다.
일수는 누적되지 않지만 좌석은 누적됩니다	갱신은 현재 권한을 대체합니다. 구매 시점부터 기간이 새로 시작되며 남은 일수는 이월되지 않으므로 기간이 끝날 무렵 갱신하세요. Team에서는 추가 Team 라이선스가 자기 기간만큼 좌석을 더하며, 팀은 마지막으로 유효한 라이선스가 끝날 때까지 유지됩니다.

폴더 안의 두 문서

[LICENSE](#) (전체 약관)와 [PRIVACY.md](#) (개인정보 보호 정책). BugIt을 활성화하고 사용함으로써 LICENSE에 동의하는 것입니다.

도움받기

먼저 위의 문제 해결과 FAQ를 확인해 주세요. 대부분의 경우를 다룹니다. 그런 다음 아래 순서대로 진행하세요.

1. AI에게 고쳐달라고 요청하기

가장 좋은 첫 시도입니다. 채팅에서 문제를 설명하고 어시스턴트에게 고쳐달라고 요청하세요. 파일을 변경했고 결과가 맞다면 Keep을 클릭해 적용하세요(사용자의 컴퓨터에만 적용됩니다).

2. 상태 점검 실행하기

[Check status](#) · [Check readiness](#) · 또는 Terminal에서 [python tools/selftest.py](#).

3. 복원하기

[Restore my settings](#) 가 잘못된 변경이나 업데이트를 되돌립니다.

4. 사람에게 연락하기

가이드와 AI로도 해결되지 않을 때만: bugit.dev를 방문하거나 support@bugit.dev로 메일을 보내주세요(지원은 영어로 제공됩니다). 첫 7일 이내 설정 문제가 있으신가요? 실행되도록 도와드리거나 스토어를 통한 환불을 도와드립니다.

지원 메일에는 기밀 프로젝트 정보를 포함하지 말아주세요

모든 프로젝트는 다르며, 작업 내용의 상당 부분이 기밀일 수 있습니다. 메일을 보내실 때는 BugIt 관련 문제를 일반적인 용어로만 설명해 주세요. 프로젝트의 기밀 코드, 버그 세부 정보, 사양, 스크린샷, 데이터를 붙여넣지 마세요. Taskivator는 저희에게 전송된 기밀 정보에 대해 책임지지 않으며, 저희가 받은 기밀 정보는 무엇이든 즉시 삭제됩니다. 가능하다면 먼저 AI 어시스턴트에게 도움을 요청해 주세요. 편집기 안에서 바로 대부분의 문제를 해결할 수 있습니다.

BugIt을 선택해 주셔서 감사합니다.

깔끔한 티켓을 등록하고, 중복을 건너뛰고, 시간을 되 찾으세요. 버그 하나씩 차근차근.

BugIt QA Agent · 설정 & 사용자 가이드 · © 2026 Taskivator. All Rights Reserved. · [bugit.dev](#) · 다운로드에 포함된 LICENSE 및 PRIVACY.md 파일이 우선 적용되는 문서입니다.